WFLOW-03: Alert Notification Basics

> **Series:** WFLOW — Workflows and Alert Notifications | **Notebook:** 3 of 10 | **Created:** January 2026 | **Last Updated:** 11/11/2026

Sending Notifications with Workflows

The most common workflow use case is sending alert notifications to Slack, Microsoft Teams, and email. This notebook covers setting up connections, configuring notification tasks, and best practices for effective alerting.

Table of Contents

1. Notification Architecture
2. Setting Up Connections
3. Slack Notifications
4. Microsoft Teams Notifications
5. Email Notifications
6. Message Formatting
7. Complete Alert Workflow Example

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	`automation:workflows:write`, `automation:workflows:run` (WFLOW-01 §4)
Workflows authorization settings	`app-settings:objects:read` and `app-settings:objects:write` for connector actions; `email:emails:send` for email. Each connector's setup page lists its full set (Workflows > Settings > Authorization settings)
External requests	A host pattern for every outbound destination (Settings > General > External requests)
Prior Knowledge	**WFLOW-01** and **WFLOW-02**

Requirement	Details
External Access	Slack workspace admin or Teams channel access

1. Notification Architecture

How Workflow Notifications Work

```
Trigger (Detected Problem, Schedule, etc.)
            ↓
    Workflow Executes
            ↓
    Notification Task
            ↓
    Connection (credentials)
            ↓
    External Service (Slack, Teams, Email)
```

Components

Component	Purpose
Connection	Securely stores credentials (tokens, webhooks)
Notification Task	Sends message using connection
Message Template	Dynamic content using Jinja expressions

Supported Notification Channels

Channel	Task Type	Authentication
Slack	`dynatrace.slack:slack-send-message` (Send message); Request approval (copy its ID from the action picker)	Bot OAuth token from a Slack app
Microsoft Teams	`dynatrace.msteams:send-message`	Teams Workflows (Power Automate) webhook URL
Email	`dynatrace.email:send-email`	Built-in — sends from `no-reply@apps.dynatrace.com`; no SMTP setup
PagerDuty	Send event (Events API v2) and REST API actions — copy the exact IDs from the action picker	Events connection (routing key) for Send event; REST API key for the others

Channel	Task Type	Authentication
ServiceNow	`dynatrace.servicenow:snow-create-incident`, plus other `snow-*` actions	OAuth or Basic Auth
Jira	`dynatrace.jira:jira-create-issue`, plus other `jira-*` actions	API Token
Custom Webhook (HTTP Request)	`dynatrace.automations:http-function`	Credential Vault (Basic or Token)

The Slack Send message, Teams, Email, ServiceNow, Jira and HTTP Request identifiers were checked against Dynatrace's published workflow samples and connector templates (10/02/2026, re-checked 10/06/2026). The PagerDuty and Slack Request approval identifiers do not appear there, so copy them from the action picker rather than typing them. Once an action has run, the platform records its app and function in `dt.system.events` (`dt.automation_engine.action.app` / `.action.function`), so a tenant can confirm them too. The YAML in this series uses these identifiers and the actions' real input names (`message`, `content`, `payload`, `connectionId`). It is simplified in two places: connection names stand in for connection IDs, and named `conditions:` lists stand in for each task's `conditions` (`states` / `custom`). Export a workflow built in the editor to get the exact document shape.

> **Sources:** threat-detection-notification-sender.yaml (Dynatrace GitHub) — *"action: dynatrace.slack:slack-send-message"*, *"action: dynatrace.msteams:send-message"*; wftpl_sample_servicenow_incident_man.yaml (Dynatrace GitHub) — *"action: dynatrace.servicenow:snow-create-incident"*; Workflow samples action catalog (Dynatrace GitHub); PagerDuty Connector actions (DT docs) — *"Trigger, acknowledge, or resolve an alert in PagerDuty using the Events API v2"*; Email (DT docs).

2. Setting Up Connections

Connections store credentials separately from workflows for security and reusability.

Accessing Connections

1. Open **Settings**
2. Select **Connections**, then the tool (Slack, Microsoft Teams, ServiceNow, …)
3. Select **Connection** to add one

Dynatrace's connector pages give this path as *Settings* > *Connections* followed by the tool — for example *Connections* > *Slack* or *Connections* > *Microsoft Teams*.

Creating a Connection

1. Open the connector (step 2 above) and select **Connection**

2. Enter the credential the connector asks for (a Slack bot token, a Teams webhook URL, …)
3. Allow the destination host under **Settings > General > External requests**
4. Name the connection (e.g., `slack-production-alerts`)
5. Save

Connection Best Practices

Practice	Description
Descriptive names	`slack-prod-oncall` not `slack1`
Environment separation	Different connections for prod/staging
Least privilege	Minimum required scopes
Regular rotation	Update tokens periodically

3. Slack Notifications

Setting Up the Slack Connection

The Slack connection takes a bot token from a Slack app you create: *"Your Dynatrace Slack Connector requires an OAuth token to authorize sending messages to Slack."*

Setup:
1. **Allow the host.** Settings > General > External requests: add a host pattern for `slack.com`.
2. **Grant Workflows the connector permissions.** Workflows > Settings > Authorization settings: enable `app-settings:objects:read`, `app-settings:objects:write` and the `state:app-states:*` and `state:user-app-states:*` permissions the setup page lists.
3. **Create the Slack app** at https://api.slack.com/apps, from an app manifest. To send messages to channels, the minimal bot scopes are `channels:read`, `groups:read` and `chat:write`. Add `chat:write.public` to post to public channels the bot has not joined. Approvals, reactions and file attachments need more scopes, listed on the setup page.
4. **Install** the app to the workspace and copy the **Bot User OAuth Token**.
5. **Create the connection.** Settings > Connections > Slack > **Connection**: name it and paste the token in **Bot token**.

> **There is no webhook option on the Slack connection.** If you must post through a Slack incoming webhook, that is an HTTP Request task (`dynatrace.automations:http-function`) to the webhook URL, outside the Slack connector, with the webhook host allowed under External requests.
>
> **Sources:** Set up Slack Connector (DT docs) — the External requests and Authorization settings steps and the minimal manifest; Slack Connector actions (DT docs) — scopes per action.

Basic Slack Message Task

```
name: send_slack_alert
action: dynatrace.slack:slack-send-message
input:
  connection: slack-production      # the Slack connection's ID in an exported workflow
  channel: "#alerts-production"
  messageFormat: slack_format
  message: |
    :rotating_light: *Problem Detected*

    *Title:* {{ event()["event.name"] }}
    *Category:* {{ event()["event.category"] }}
    *Started:* {{ event()["event.start"] }}

    &lt;{{ problem_link() }}|View in Dynatrace&gt;
```

Slack Message with Blocks

For richer formatting, put a Block Kit payload in `message`. The Send message action has no separate `blocks` input: with `messageFormat: slack_format`, *"Slack Markdown or Slack Block Kit inputs are processed"* from the message field.

```
input:
  connection: slack-production
  channel: "#alerts"
  messageFormat: slack_format
  message: |
    {
      "blocks": [
        { "type": "header",
          "text": { "type": "plain_text", "text": "{{ ':red_circle:' if
(event().get('event.severity') | int(5)) &lt;= 1 else ':large_orange_circle:' }} {{ event()
['event.category'] }} Alert" } },
        { "type": "section",
          "fields": [
            { "type": "mrkdwn", "text": "*Problem:*\n{{ event()['event.name'] }}" },
            { "type": "mrkdwn", "text": "*Status:*\n{{ event()['event.status'] }}" } ] },
        { "type": "section",
          "fields": [
            { "type": "mrkdwn", "text": "*Started:*\n{{ event()['event.start'] }}" },
            { "type": "mrkdwn", "text": "*ID:*\n{{ event()['display_id'] }}" } ] },
        { "type": "actions",
          "elements": [
            { "type": "button", "text": { "type": "plain_text", "text": "View Problem" },
"url": "{{ problem_link() }}" } ] }
      ]
    }
```

The expressions resolve before the JSON reaches Slack, so a problem title that contains a double quote breaks the payload. Build the layout in Slack's Block Kit Builder and send one test message before you rely on it.

> **Sources:** Slack Connector actions (DT docs) — *"If messageFormat is slack_format , Slack Markdown or Slack Block Kit inputs are processed."*; Workflow samples action catalog (Dynatrace GitHub) — *"Slack messages can be plain text/Slack Markdown, or a JSON Block Kit payload"*.

4. Microsoft Teams Notifications

> **Office 365 connectors no longer work.** Microsoft's final schedule: *"Rollout begins : May 18, 2026 Rollout completes : May 22, 2026 After these dates, Office 365 Connectors will no longer function."* An Office 365 incoming-webhook URL in a Dynatrace connection stops delivering. The Dynatrace Microsoft Teams connector takes a Teams **Workflows** (Power Automate) webhook URL instead.

Setting Up the Teams Connection

1. **Create the webhook in Teams.** Open the channel's context menu, select **Workflows**, and choose **Send webhook alerts to a channel**. Save, then select **Copy webhook link**.
2. **Allow the host.** Settings > General > External requests: add a host pattern for the webhook's domain. Dynatrace's example is `*.api.powerplatform.com`.
3. **Create the connection.** Settings > Connections > Microsoft Teams > **Connection**: name it and paste the URL in **Webhook URL**.

Messages arrive under the name of the user who created the webhook. If an existing connection's URL contains `logic.azure.com`, the Teams connector page explains how to replace it with the flow's updated URL.

Basic Teams Message Task

```
name: send_teams_alert
action: dynatrace.msteams:send-message
input:
  connectionId: teams-production    # the Teams connection's ID in an exported workflow
  messageFormat: dynatrace_markdown # converts each Markdown element to a card element
  message: |
    **Problem Detected**

    **Title:** {{ event()["event.name"] }}
    **Category:** {{ event()["event.category"] }}
    **Started:** {{ event()["event.start"] }}

    [View in Dynatrace]({{ problem_link() }})
```

Teams Adaptive Card

For richer formatting, put Adaptive Card JSON in `message` with the default `messageFormat` (`msteams_format`), or pick a predefined card in `selectTemplate`.

```
input:
  connectionId: teams-production
  message: |
    {
      "type": "AdaptiveCard",
      "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
      "version": "1.4",
      "body": [
        { "type": "TextBlock", "text": "{{ event()['event.category'] }} Alert", "size":
"Large", "weight": "Bolder",
          "color": "{{ 'Attention' if (event().get('event.severity') | int(5)) &lt;= 1 else
'Warning' }}" },
        { "type": "FactSet", "facts": [
            { "title": "Problem", "value": "{{ event()['event.name'] }}" },
            { "title": "Status", "value": "{{ event()['event.status'] }}" },
            { "title": "Started", "value": "{{ event()['event.start'] }}" } ] }
      ],
      "actions": [
        { "type": "Action.OpenUrl", "title": "View in Dynatrace", "url": "{{ problem_link()
}}" }
      ]
    }
```

Use Dynatrace expressions for the dynamic values, as above. The docs say *"We don't support Adaptive Cards Template Language templating"*.

> **Sources:** Microsoft Teams Connector (DT docs) — setup steps, `messageFormat` (*"Must be msteams_format or dynatrace_markdown"*) and `selectTemplate`; Retirement of Office 365 connectors within Microsoft Teams (Microsoft 365 Developer Blog); threat-detection-notification-sender.yaml (Dynatrace GitHub) — *"action: dynatrace.msteams:send-message"*; Workflow samples action catalog (Dynatrace GitHub).

5. Email Notifications

Email Options

Option	Setup	Best For
Send email action (`dynatrace.email:send-email`)	No setup — sends from `no-reply@apps.dynatrace.com`; the workflow needs the `email:emails:send` permission	Most notifications

Option	Setup	Best For
Corporate mail relay	Not an option of the Send email action — call the relay's HTTP API from an HTTP Request or JavaScript task	A corporate sender address is required
SendGrid/SES	HTTP Request task to the provider's API	High volume, HTML templates

The two HTTP routes need the provider's host allowed under **Settings > General > External requests**. A host on a private network, such as a corporate relay, is reached through EdgeConnect (WFLOW-94).

The Send email action has two limits: *"For To , Cc , and Bcc fields the number of email addresses is restricted to 10 per field."* and *"Trial environments are prohibited to send emails with Email ."*

Basic Email Task

```
name: send_email_alert
action: dynatrace.email:send-email
input:
  to:
    - "oncall@company.com"
    - "platform-team@company.com"
  cc: []
  bcc: []
  subject: "[{{ event()['event.category'] }}] {{ event()['event.name'] }}"
  content: |
    A problem has been detected in your Dynatrace environment.

    Problem Details:
    ----------------
    Title: {{ event()["event.name"] }}
    Category: {{ event()["event.category"] }}
    Status: {{ event()["event.status"] }}
    Start Time: {{ event()["event.start"] }}
    Problem ID: {{ event()["display_id"] }}

    Affected Entities:
    {{ (event().get("smartscape.affected_entities") or []) | map(attribute="name") | join(",
") or event()["affected_entity_ids"] | join(", ") }}

    Root Cause:
    {{ (event().get("root_cause.smartscape_entity") or {}).get("name", "not yet determined")
}}

    View this problem:
    {{ problem_link() }}
```

Formatted Email

The root cause is read with `.get()` because most problems do not carry one: on a validation tenant, 127 of 540 problems over 24 h had `root_cause.smartscape_entity` (10/06/2026), and a bracket read of a

missing field fails the task with *Undefined variables*. The older `root_cause_entity_id` is deprecated and just as often absent. The affected-entities line follows the same rule: it prints the names from `smartscape.affected_entities` and falls back to the deprecated `affected_entity_ids` when that field is empty — see the callout in §6. Enabling **Wait for root cause analysis** on the trigger (WFLOW-02 §2) addresses the early-lifecycle gap but does not guarantee a root cause: *"Dynatrace Intelligence does not populate a root cause for every problem, particularly early in the lifecycle or for externally ingested events."*

The `content` field takes markdown: bold, headings, lists, tables and links. It does not take HTML, so an HTML template arrives as literal tags, and there is no content-type input. For an HTML template, send through SendGrid or SES from an HTTP Request task (table above).

```
input:
  to: ["oncall@company.com"]
  cc: []
  bcc: []
  subject: "[{{ event()['event.category'] }}] {{ event()['event.name'] }}"
  content: |
    ## {{ event()['event.category'] }} alert

    | | |
    |---|---|
    | **Problem** | {{ event()['event.name'] }} |
    | **Status** | {{ event()['event.status'] }} |
    | **Started** | {{ event()['event.start'] }} |

    [View in Dynatrace]({{ problem_link() }})
```

> **Sources:** Email (DT docs) — *"It doesn't offer support for HTML."*; threat-detection-notification-sender.yaml (Dynatrace GitHub) — *"action: dynatrace.email:send-email"*.

6. Message Formatting

Jinja Expression Reference

Expression	Result	Use
`{{ event()["event.name"] }}`	Problem title	Main content
`{{ event()["event.category"] }}`	AVAILABILITY/ERROR/SLOWDOWN/…	What kind of problem
`{{ event().get("event.severity") | int(5) }}`	1 (most severe) … 5	Color coding

Expression	Result	Use
`{{ (event().get("smartscape.affected_entities") or []) | map(attribute="name") | join(", ") }}`	Comma-separated entity names	Show all affected
`{{ event()["affected_entity_ids"] | join(", ") }}`	Comma-separated classic IDs	Fallback only — deprecated
`{{ event()["event.start"] }}`	ISO timestamp	When started
`{{ problem_link() }}`	Full URL	Link to problem (Problem trigger only)

> **Deprecated problem fields.** The upgrade guide marks the classic entity fields deprecated: *"root_cause_entity_id and root_cause_entity_name are deprecated in favor of root_cause.smartscape_entity , and affected_entity_ids in favor of smartscape.affected_entities . Build new payloads against the Smartscape fields."* `affected_entity_ids` still works and is the better-populated field: over 7 days on a validation tenant (10/14/2026) it was set on 5,014 of 5,014 problem records and `smartscape.affected_entities` on 4,056, so the templates in this notebook read `smartscape.affected_entities` first and fall back to it, as in the email template in §5. **WFLOW-02 §8** maps every payload field and **WFLOW-05 §3** has the population counts.
>
> **Sources:** Upgrade guide — alerting and notifications (DT docs) — *"smartscape.affected_entities Record array; each element has id , type , name".*

`event.severity` is `stable` in the semantic dictionary (read 11/11/2026) and is a 1–5 scale; the problem record carries no `CRITICAL` / `HIGH` strings. `int(5)` treats a missing severity as least severe — see WFLOW-04 §3 for why that default matters from SaaS 1.348.

Conditional Formatting

```
{% set sev = event().get("event.severity") | int(5) %}
{% if sev &lt;= 1 %}
:red_circle: CRITICAL ALERT
{% elif sev == 2 %}
:large_orange_circle: MAJOR ALERT
{% else %}
:large_yellow_circle: {{ event()["event.category"] }} ALERT
{% endif %}
```

Severity Emoji Map

`event.severity`	Slack Emoji	Teams Color
1 (Critical)	`:red_circle:`	`Attention`
2 (Major)	`:large_orange_circle:`	`Warning`
3 (Minor)	`:large_yellow_circle:`	`Accent`
4 (Warning)	`:large_blue_circle:`	`Good`

Level names follow the Davis severity scale (see WFLOW-04 § 3); Warning and Informational events never open a problem, so a problem-triggered workflow sees levels 1–3.

Inline Severity Mapping

```
{{ {1: ":red_circle:", 2: ":large_orange_circle:", 3: ":large_yellow_circle:", 4:
":large_blue_circle:"}.get(event().get("event.severity") | int(5), ":white_circle:") }}
```

7. Complete Alert Workflow Example

A workflow that sends the same problem to Slack, Teams, and email. Tasks are keyed by name and the trigger block uses the exported shape (WFLOW-02 §2); export a workflow built in the editor for the exact document.

Workflow Configuration

```
name: production-alert-notifications
description: Send alerts to all channels for production problems

trigger:
  eventTrigger:
    isActive: true
    triggerConfiguration:
      type: davis-problem
      value:
        triggerOn: open
        categories:
          availability: true
          error: true
          slowdown: true
          resource: true
        entityTagsMatch: all
        entityTags:
          env:
            - prod
        analysisReady: true          # Wait for root cause analysis

tasks:
  slack_notification:
    name: slack_notification
    action: dynatrace.slack:slack-send-message
    input:
      connection: slack-production
      channel: "#alerts-production"
      message: |
        {{ {1: ":red_circle:", 2: ":large_orange_circle:", 3:
":large_yellow_circle:"}.get(event().get("event.severity") | int(5), ":white_circle:") }} *
{{ event()["event.category"] }} Problem*

        *{{ event()["event.name"] }}*

        • *Status:* {{ event()["event.status"] }}
        • *Started:* {{ event()["event.start"] }}
        • *ID:* {{ event()["display_id"] }}

        &lt;{{ problem_link() }}|:mag: View Problem&gt;

  teams_notification:
    name: teams_notification
    action: dynatrace.msteams:send-message
    input:
      connectionId: teams-production
      messageFormat: dynatrace_markdown
      message: |
        **{{ event()["event.category"] }} Problem**

        **{{ event()["event.name"] }}**

        - **Status:** {{ event()["event.status"] }}
```

```
        - **Started:** {{ event()["event.start"] }}
        - **ID:** {{ event()["display_id"] }}

        [View Problem]({{ problem_link() }})

  email_notification:
    name: email_notification
    action: dynatrace.email:send-email
    input:
      to: ["platform-oncall@company.com"]
      cc: []
      bcc: []
      subject: "[{{ event()['event.category'] }}] {{ event()['event.name'] }}"
      content: |
        A {{ event()["event.category"] }} problem has been detected.

        Problem: {{ event()["event.name"] }}
        Status: {{ event()["event.status"] }}
        Started: {{ event()["event.start"] }}

        View: {{ problem_link() }}
```

Task Execution

All three tasks have no predecessor, so they start in parallel when the trigger fires. A task that does have predecessors waits for them: *"By default, a task will run if its predecessors ended successfully."*

Cost: this is a standard workflow

Three tasks make this a **standard** workflow, which consumes workflow hours for as long as it exists (WFLOW-01 §3.5). For a single destination, a simple workflow with one task costs no workflow hours. The upgrade guide keeps standard workflows for cases like this one, *"when one filter must fan out to several destinations in a single configuration."*

> **Sources:** Monitor workflow executions (DT docs); Upgrade guide — alerting and notifications (DT docs); Automation Workflow consumption (DT docs).

Monitor Your Notification Workflows

```
// Notification workflow executions
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is
no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling
— those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED ·
SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it,
otherwise a
//                                        single execution is counted once as RUNNING and
again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| filter in(dt.automation_engine.action.app, {"dynatrace.email", "dynatrace.slack",
"dynatrace.servicenow"})
| summarize executions = count(), by:{dt.automation_engine.action.app,
dt.automation_engine.state}
| sort executions desc
```

```
// Notification task success rates
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is
no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling
— those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED ·
SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                    true only on terminal records — filter on it,
otherwise a
//                                        single execution is counted once as RUNNING and
again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {total = count(), succeeded = countIf(dt.automation_engine.state == "SUCCESS")},
by:{dt.automation_engine.action.function}
| fieldsAdd success_pct = round(succeeded * 100.0 / total, decimals: 1)
| sort total desc
| limit 20
```

```
// Failed notification tasks
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is
no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling
— those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED ·
SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                    true only on terminal records — filter on it,
otherwise a
//                                        single execution is counted once as RUNNING and
again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.action.function, dt.automation_engine.task.name,
dt.automation_engine.state_info
| sort timestamp desc
| limit 25
```

Next Steps

Now that you can send basic notifications, learn advanced patterns:

Recommended Path

1. **WFLOW-04: Advanced Notification Routing** - Conditional routing by severity, team, time
2. **WFLOW-05: PagerDuty & ServiceNow** - Incident management integration
3. **WFLOW-06: Custom Notification Templates** - Rich formatting and dynamic content

Key Takeaways

- **Connections** store credentials securely and are reusable

- **Slack** connections take a bot OAuth token from a Slack app; there is no webhook option
- **Teams** connections take a Teams Workflows (Power Automate) webhook URL; Office 365 connectors stopped working in May 2026
- **Email** sends from `no-reply@apps.dynatrace.com`; for another sender, call a mail provider's API from an HTTP Request task
- Every outbound destination needs an **External requests** host pattern
- **Jinja expressions** enable dynamic message content; read optional fields such as the root cause with `.get()`
- Tasks with no predecessor run in **parallel**; more than one task makes a **standard** workflow, billed in workflow hours

---

Summary

In this notebook, you learned:

- Notification architecture: triggers → tasks → connections → external services
- How to set up connections for Slack, Teams, and email
- Basic and advanced Slack message formatting
- Microsoft Teams notifications with Adaptive Cards
- Plain-text and markdown email notifications (the Send email action does not render HTML)
- Jinja expressions for dynamic content
- A complete multi-channel alert workflow

References

- Notification actions umbrella (DT docs)
- Slack action (DT docs)
- Set up Slack Connector (DT docs)
- Retirement of Office 365 connectors within Microsoft Teams (Microsoft 365 Developer Blog)
- Microsoft Teams action (DT docs)
- Email action (DT docs)
- PagerDuty action (DT docs)
- Workflow reference / Jinja expressions (DT docs)
- Alerting and notifications umbrella (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.